



Computer (& AI) Methods for Structural Engineering (DISC 5203)

Fall 2026

Homework 2 - Due date: 06/10/2026 (1st) and 13/10/2026 (final)

Homework Assignment 2: From Slab to Beam — Tributary Areas, Line Loads and a Two-Dimensional Subframe

Context

This assignment continues the larger project begun in Homework 1. Where HW1 read a floor plan *down into the columns* — tributary areas of vertical elements and the axial load each one carries — HW2 reads the same plan *across the slab and onto the beams*. It is the first stage of the two-dimensional subsystem that later homeworks build up (coupled walls, rigid offsets, diaphragms).

It has the same two purposes as HW1: to apply structural-engineering fundamentals — here slab-to-beam load transfer, the shape of a line load, and the difference between a beam *on its own* and that same beam *in a frame* — and to gain hands-on experience with an AI agent in a real project workflow, this time with the **speech** → **structural model** thread: you transcribe a short, plain-language structural *briefing* into a solver input file (here, OpenSees), the agent builds and runs it, and you judge the result against a bound you set *before* the solve.

The course strongly encourages the use of an AI agent tool rather than a plain chatbot. The agent should read files, write scripts, run code, check results, and iterate autonomously within the project folder. You are responsible for predicting what the answer should be, checking the agent's work, and signing a verdict.

Input

You continue to use the **same floor plan image as HW1**, selected by the last digit of your student ID number. Do **not** re-derive the floor outline, the columns, or the *column* tributary areas from HW1 — you build on that Git state and reuse all of them.

This is the distinction the whole assignment turns on, and it is worth stating plainly:

- **HW1 (column tributary areas)**. Each slab panel was split between the supporting **columns**, and the load was read *down* from the slab to the vertical elements, giving one tributary area per column and the axial load in each column.

- **HW2 (beam tributary areas).** The *same* panels are now split between the supporting **beams**, and the load is read *across* the slab *onto* the beams. This gives a *tributary area per beam* and, in Task 2, a *line load* on each beam — a quantity HW1 never produced. The same physical slab load ends up on different members, and you must show the path both ways: slab → beams here, and beams → columns for the column axial loads of HW1.

So the new work is the load path *between* the slab and the beams, not a re-computation of the column tributary areas.

Assume the supplied floor plan is a typical floor of the same **5-storey building** as HW1: the same floor on all five floors, storey height 3 m, a live (imposed) area load of 5 kN/m^2 , and, where the plan does not give it, a slab thickness $h = 150 \text{ mm}$. Concrete density: take 25 kN/m^3 for the slab self-weight unless the plan states otherwise; add the superimposed finishes only if the plan gives a figure. State every load assumption you make.

For the two beams of Task 3, your plan (one of the three figures below, chosen by the last digit of your student ID) carries **two red dashed boxes**: a **vertical** dashed rectangle around a vertical beam that runs in the grid direction, and a **horizontal** dashed rectangle around a horizontal beam. Read the two marked beams from your own figure (label the vertical one **Beam A** and the horizontal one **Beam B**, or use whatever lettering the marking expects). The two beams are chosen so that one is a **peripheral (edge) beam**, supporting the slab on **one side only**, and the other is an **interior (continuous) beam**, supporting the slab on **both sides** and running over an intermediate support, so that the two line loads come out with *different shapes* and the "pure beam" and "subframe" models of the continuous one differ. Do not assume which box is which — identify them from the plan.

Tasks

Using an AI agent, work through the following steps and document your methodology.

1. Tributary areas of the beams

This task produces the **beam** tributary areas — distinct from, and *not* a repeat of, the **column** tributary areas of HW1.

1. Identify every beam in the floor plan and report its endpoints, span, depth (where given), and the panel(s) it borders.
2. Determine the **tributary (influence) area of each beam** from the slab that bears on it. This is *not* the column tributary area of HW1: in HW1 the load was read *down* from the panel onto the supporting **columns** (a tributary area per column); here the load is read *across* the panel onto its supporting **beams**, split along the mid-line between the beams, constructed from the 45° bisectors at the panel corners.
3. Note any beam that carries load on **one side only** (a perimeter beam) and any beam that carries it on **both sides** (an interior beam); a one-sided beam and a two-sided beam of the same span carry very different loads.

2. Line load on each beam — a table

4. Convert each beam's tributary load into a **line load** (force per unit length, kN/m) along its length: dead load from the slab self-weight plus any superimposed load, and live load, separately, then

the total. Where the tributary load is a triangle or trapezium, give the **equivalent uniform load** and label which moment it is equivalent to.

5. Produce a **summary table for every beam** containing at least: beam ID, location (column line / grid), span(s), depth, tributary width or area, the load *shape*, dead + live + total line load (kN/m), and the equivalent uniform load used for moment. The table is the deliverable; show a worked example for one beam fully (equations, intermediate values, a plot of the load diagram).

3. Two beams, two models — pure beam *and* one-floor subframe

For the **two beams marked with the red dashed boxes** in FloorPlan png file according to your student ID number (Beam A and Beam B defined above), and using the **line loads you found in Task 2**:

6. **Verify a bound before modelling.** For each beam, by hand and *before* you run the agent, write down the order of magnitude of its peak bending moment — for a simply supported span $\approx qL^2/8$, for a continuous span a fraction of that; state the shape you expect (hogging over an intermediate support, sagging at mid-span) and the sign. This prediction is the check you will hold the agent's result to.
7. **Model A — pure (standalone) beam.** Build a single element representing just the beam under its own line load, on simple supports (or, for Beam B, fixed/continuous over its supports as the plan implies). Solve and plot the **deformed shape** and the **axial, shear, and bending-moment diagrams (N, V, M)**. Check the moment against your hand bound.
8. **Model B — one-floor subframe.** Build a small **two-dimensional grid** containing the beam and the beams and/or columns immediately around it on that floor — enough that Beam B's supports can spread the load to their neighbours (at least a one-bay strip in the perpendicular direction, with the supporting column lines). Apply the **same line loads**. Solve and plot, for the central beam, the deformed shape and the N, V, M diagrams, and report the **support (column) reactions**.
9. **Check the deformed shapes by hand.** For each model (A and B), and for each beam (Beam A and Beam B), sketch the expected **deformed shape by hand** — its curvature and where the peaks and inflection points sit — and place it beside the shape the solver produced. Comment on the **deformed shape of the central beam in the subframe** and on the whole subframe: for Beam B, does the beam *kink* at its supports, and does the surrounding subframe deflect as one? A diagram that does not match your sketch by eye is a symptom of a wrong boundary condition, a sign error, or a mis-scaled section — the cheapest way to catch each.
10. **Compare the two models.** State, in one paragraph, how and why the **deformed shape, bending moment, and reactions** of the two models differ. A beam on its own differs from the beam in a frame by the **continuity over the supports**, the **perpendicular grid spreading the load** to neighbouring members, and the **moment the subframe attracts at its supports**. Then say which model is the **defensible** one for the beam's own design, and *why* — a model that answers a different question than the one you asked is not wrong; it is the wrong model for the job.
11. **Build the input files with an AI agent (speech → model).** Give the agent a plain-language structural briefing (a short paragraph, or a transcript of a spoken briefing) that describes each model — the spans, the supports, the loads, the section dimensions and material — **not** a finished script. Let the agent transcribe that briefing into the **OpenSees input file**, run it, and plot the response diagrams. Record the **prompt(s) you gave the agent** and the **agent's iteration** (the briefing, each revision, and the

human checks that gated them). This is the W5 speech → model thread; the human-in-the-loop gate is: *confirm the deck, freeze the loads, sign off the briefing→models mapping.*

12. **Cross-check the beam model's axial forces against HW1's column axials.** From the **one-floor subframe (Model B)** of the last step, read off the **beam model's axial force** at their supports (the column top, i.e. the \$N\$ in the beam's N-diagram just at the grid node — for the supporting column, \$N\$ is the same quantity read on the column's bottom end) and compare, member by member, with the **column axial forces you computed in HW1** (Task 4, the column tributary areas). This is the consistency link between the two deliverables: both numbers should describe the **same storey reaction at the same grid point**, so they must be of the same order of magnitude and sign. Report a small table — grid point, HW1 column axial (from column tributary area × storey load), HW2 beam axial (from the subframe reaction), and the percentage difference — then explain any gap. A large, unexplained gap is a symptom of one of: (a) a **tributary-area error** in one of the two (the load split to the column), (b) a **missing or doubled load** in the subframe, (c) a **sign or units slip** between the two readings, or (d) a **stiffer subframe** in HW2 spreading load that HW1's single-storey tributary picture did not — i.e. the two models are *not answering the same question*. A close match (within a few percent on the order of magnitude, exact on the sign) is the positive result. Where they do **not** match, say which one is to be trusted and why — generally the **full subframe result**, as it carries the true load path, and the HW1 tributary number as the *independent estimate* it is checked against.

Deliverables

Submit a Markdown (.md) report containing:

- A description of your methodology for reading the load onto the beams, with the 45° tributary construction shown.
- The beam census (IDs, locations, spans, depths) and the **tributary-area** results.
- The **line-load table** for every beam, with one beam worked fully (equations, intermediates, load-diagram plot).
- For Beam A and Beam B: the **pure-beam** and **one-floor subframe** OpenSees input files, their **response diagrams** (deformed shape and N, V, M, embedded in the Markdown), the **hand-drawn sketches of the deformed shapes** placed beside each solver output, the **comparison of the two models**, and the **hand bound** you set beforehand.
- The **subframe-vs-HW1 column-axial cross-check** (Task 12): the per-grid-point table and the percentage-difference plot, with your explanation of any gap.
- The **agent prompt(s) and iteration log** for building the two input files from the briefing.
- Units, equations, intermediate values, and verification checks sufficient to reproduce and defend the results.

Also submit the project repository, **extended from your HW1 Git state**, including the OpenSees input files and scripts, configuration files (CLAUDE.md or equivalent, skills.md, and any hooks), the response-diagram plots, and any intermediate outputs. The subframe must be *built onto* the HW1 work, not re-derived from scratch.

Required Tools & Workflow

- Use an **AI agent tool** (Claude Code, OpenCode, DeepSeek Harness, or an equivalent harness) that can read/write files, execute scripts, and iterate on results within a project environment.

- For the agent to build the OpenSees models, give it a **plain-language briefing, not a script**: state the geometry, supports, loads and section in words (or as a transcript), and have the agent transcribe that into the solver input. Keep the briefing and the generated file both in the repository.
- Set up or extend a project structure that may include a `CLAUDE.md` (or equivalent) guidance file, a `skills.md` for reusable procedures, and hooks for deterministic actions.
- Use **Git** to track changes and extend the HW1 repository; do not start a fresh tree.
- Use **OpenSees** (OpenSeesPy) for the two model types; the course's W5 examples (`A1_continuous_beam.py`, `A2_frame_lateral.py`) are the starting point.
- Review the agent's code and results yourself. Verify calculations, units, model interpretation, and structural logic before the model is accepted.

Suggested Workflow

1. Start from your **HW1 Git state**; do not re-derive the columns or the floor outline.
2. Add the beams to the model: extract their geometry and load the slab onto them with the 45° tributary construction.
3. Build the line-load table step by step with the agent, checking one beam by hand first.
4. Write the **briefing** for Beam A and Beam B; let the agent transcribe it into two OpenSees input files (pure beam, then subframe) and run them.
5. Predict the peak moment by hand **before** the solve; then plot and check the agent's diagrams against it.
6. Commit to Git after each meaningful step and use `git diff` to inspect the changes.

Rules & Expectations

- **Individual work.** The assignment must be completed individually so that you fully understand every step of the process.
- **You must be able to defend your submitted work.** Be prepared to explain your methodology, justify assumptions, and answer questions about both the engineering content and the AI-tooling workflow — including *which* of the two models is the right one for the beam and why.
- **Verify all AI outputs.** AI models can hallucinate or make errors with high confidence. The three commonest errors in this assignment, in order, are: a line load with the wrong *shape* (triangle vs uniform, one-sided vs two-sided); a support missing a fix or a node mis-tagged; and section/property units (e.g. depth in cm, 4 cm^4 vs 4 m^4). You are responsible for checking each.
- The **known input** (the same floor plan as HW1, with the two marked beams) is the major component of the grade.
- An **unknown input** — extending the line-load and subframe method to a *different, more complex* floor plan of the same kind (a different group's plan, or a plan with irregular bays) — will account for approximately a third of the grade for this homework. Your methodology must therefore generalise beyond one plan. This is the course's **generalisation / distractor** criterion: a memorised procedure is not enough.

Deadline & Checkpoint

Milestone	Due
Checkpoint - partial results and evidence of using the tools	06/10/2026

Milestone	Due
Final submission - full Markdown report, the two OpenSees models and their diagrams, and the extended Git repository	13/10/2026

What to Submit

1.

The final **Markdown report** containing your methodology, results, calculations, the two models and their diagrams, and the agent prompts.
2.

The **project repository** (Git, extended from HW1) with the OpenSees input files, configuration files (CLAUDE.md, skills.md, hooks, etc.), supporting scripts, plots, and intermediate outputs.
3.

Be prepared to **demonstrate and discuss** your workflow in a follow-up session, including how you transcribed the briefing into the input files and how you judged the result.

What Comes Next

The subframe in this assignment is the first two-dimensional subsystem you build. The next homeworks extend it: a **coupled wall** and a **rigid offset** in a two-dimensional subsystem (W6), and the **in-plane** behaviour of a floor — diaphragms, stiffness distribution, stiffness centre — in HW3. The line-load and subframe method you develop here carries straight forward: the same beams on the same grid, read at a higher resolution.

Note: The three floor-plan figures below (one per group) already show the two red dashed boxes; mark up an additional copy if you need to, and the OpenSees environment walkthrough will be provided in the next session.

hint: Rules for Slab-to-Beam Load Transfer (from L04)

The deck gives the rules in five parts. Read each panel on its own terms; the type is a statement about the load path, *not* about the panel shape.

01 — Classify first: count supported edges, then look at the span ratio

A slab is one of three types by **how many edges support it and how far apart they are**:

Type	Supported edges	Load path	Typical example
Cantilever	1 edge	all load to that one edge	balcony, canopy, overhang
One-way	2 opposite edges	one direction only; a row of parallel 1 m beams	long thin panel <i>or</i> any panel with the perpendicular edges unsupported
Two-way	4 edges, $\alpha \leq \frac{1}{2}$	both directions; bending (and corner torsion) in both	square and near-square interior bays

The **trap is the support arrangement over the side ratio**: a panel with *only its two 4 m edges* supported spans the *long* 6 m direction and is one-way regardless of shape. $\alpha = l_y/l_x \leq 2$ is where the code accepts the long direction as negligible (you lose about 12 % of the moment by ignoring it); beyond it the long edges starve.

02 — The easy cases, by formula

Work a **1 m wide strip** spanning the load direction; n is the area load, l_x the span.

Type	Line load on the supporting beam(s)	End moment	Note
Cantilever , span l_x	$q = n, l_x$ (one side only, kN/m)	$M = n, l_x^2/2$ (hogging, tension on top)	the whole strip rides on one edge
One-way , span l_x	$q = 1/2 n, l_x$ on <i>each</i> of two beams	$M = n, l_x^2/8$ (simply supported)	the two edges <i>perpendicular</i> to the span carry no slab load
Two-way	split between all four edges (01–04 below)	—	see 03/04 and the β_v table

The free-edge cases: a free edge on the **short** side leaves the slab one-way (still $q = 1/2 n, l_x$ on the two long beams, plus an *edge strip* to tie the free edge); a free edge on the **long** side routes load by deflection-matching, n to each long beam in the ratio $\kappa_c = 5\alpha^4/(48+5\alpha^4)$ — about 35 % at $\alpha = 1.5$ rising to about 63 % at 2.0. **A free edge is not a support.**

03 — Why the load goes the short way: stiffness $k \propto 1/l^3$

Same depth and same concrete mean EI is constant, so $k \propto 1/l^3$. If $l_y = 2 l_x$ the short span is $2^3 = 8$ times stiffer and takes $8/9$ of the load. That stiffness story is what the yield-line construction of 04 merely *tabulates*; it is also why a continuous edge attracts load (a stiffer, more-restrained path — see 28 in the deck: one continuous long edge pulls 37 %, the next 26 %, then 21 %, then 15 %).

04 — Tributary areas by yield lines (45° / 60°)

Tributary areas are set by the **corner angles**, not guessed:

- **matched supports** — simple/simple or fixed/fixed — give **45°** bisectors; the panel splits into **triangles on the short edges, trapezia on the long edges**. For a square simply-supported four-edge panel that is exactly 25 % to each edge.
- **mixed supports** — one edge fixed, the opposite simple — give **60° from the fixed edge and 30° from the simple edge**, skewing load toward the continuous side.
- As α grows from 1 to 2, a **long edge grows from 25 % to 37.5 %** and a **short edge falls from 25 % to 12.5 %** — the long edges fill, the short edges starve.

Trap: a *discontinuous* edge in the β_v table is **still a support** (a load path exists across it); a genuinely *free* edge is **not**, and the table **does not apply** to a panel with a free edge (use a free-edge case instead).

05 — From tributary shape to an equivalent UDL (do this two ways and reconcile)

The tributary areas give a **non-uniform** line load — **trapezoid on long edges, triangle on short edges**. For practical design you replace it by an equivalent UDL; two routes, and **close the total-load check between them**:

1. **Coefficient route** (HKConcrete 2013, Table 6.7, Clause 6.1.3.3): $q = 0.75 \beta_{v,n,l_x}$. Pick the panel type, read two coefficients, write four line loads. **Trap: β_{vx} applies on edges of length l_y , β_{vy} on edges of length l_x ; and q always uses l_x , the short span, never l_y .**
2. **Area route**: the total load on a beam is its tributary-area $\times n$; divide by its length to get the equivalent UDL. A triangle of peak q_{peak} is equivalent, for **span moment**, to two-thirds of q_{peak} ; a trapezium by its mean ordinate.

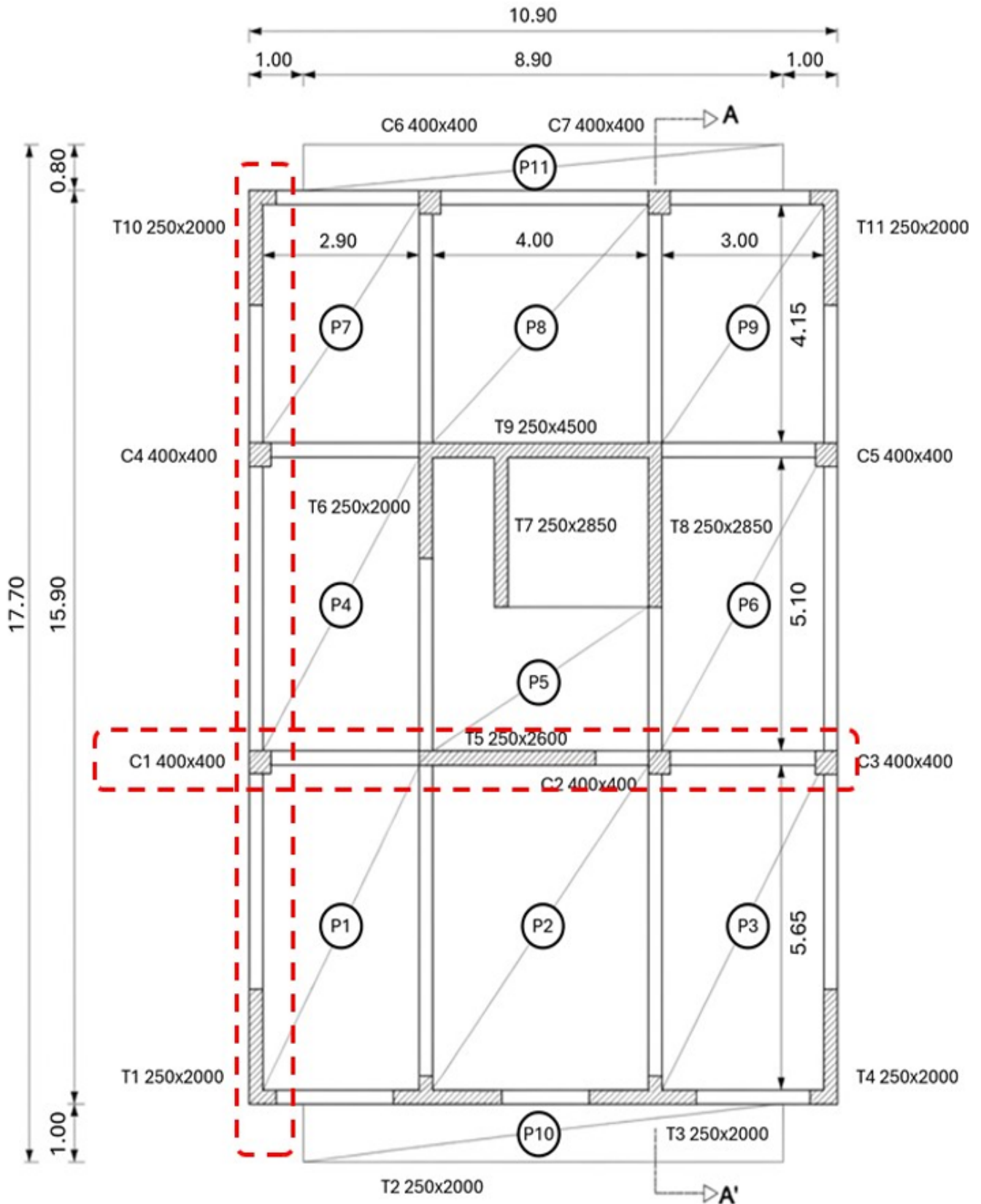
Trap: the equivalence is **by total load, not by peak intensity** — so it does not close for **local shear or for the moment diagram shape**. Label which quantity your equivalent UDL stands in for, and always reconcile route 1 against route 2 (the deck's $\alpha=1.5$, four edges discontinuous: 33.3% of $n l_x$ by area vs 34% by the table — they agree).

Two traps to name explicitly in your report: (1) β_{vx} vs β_{vy} / l_x — pick the right edge, and the formula always uses the *short span*. (2) A free edge is not a support, so Table 6.7 does not apply to it; a *discontinuous edge* is. State which type each panel is and *why*, before you compute its load.

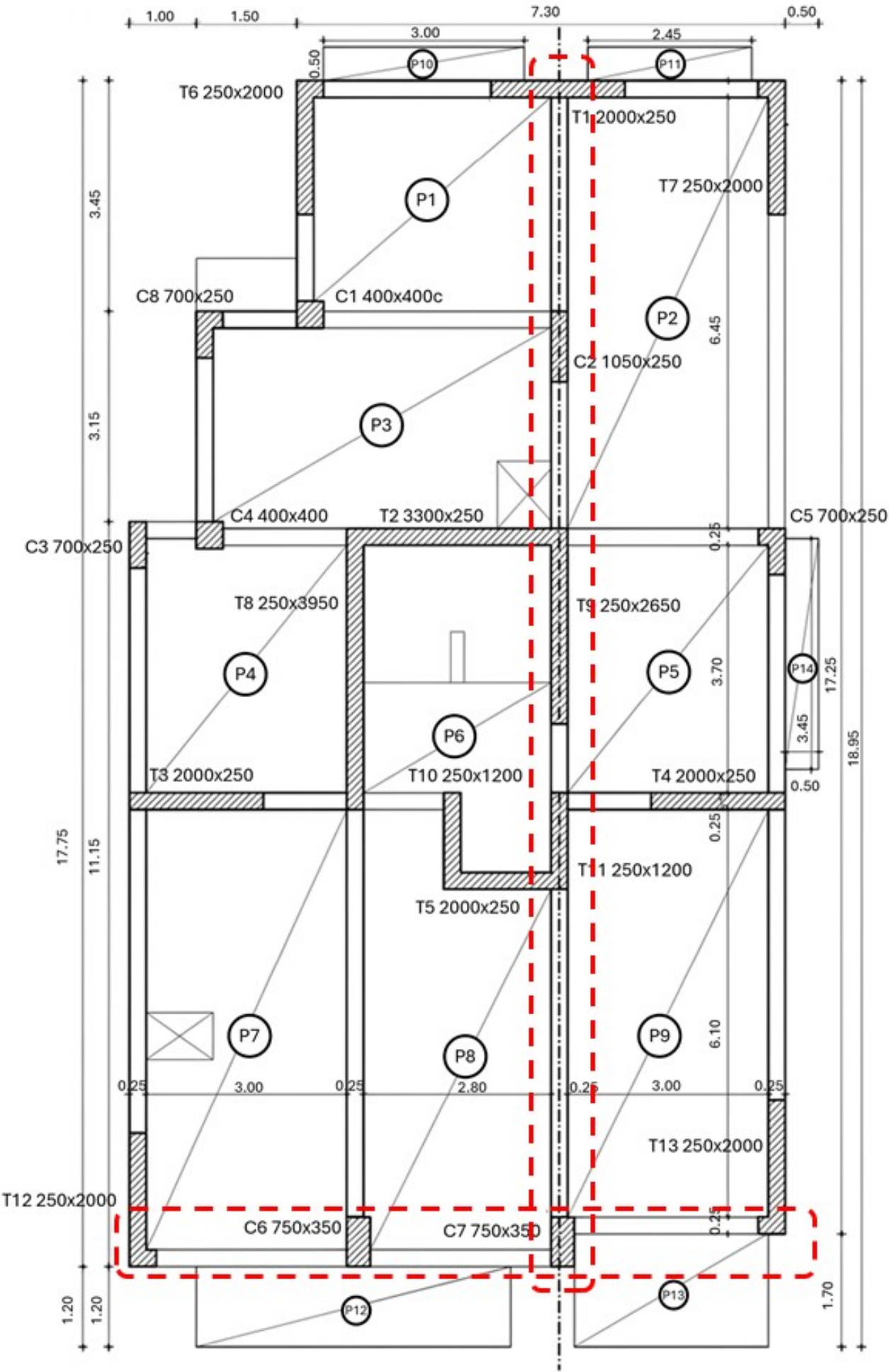
hint: OpenSees conventions for the two models (from W5)

- **Units**: kN, m, kPa — so E is entered in kN/m^2 ($30 \text{ GPa} = 30.0 \times 10^6$) and section I in m^4 . The three commonest errors are a load with the sign/wrong component, a support missing a fix, and I in cm^4 instead of m^4 .
- **Pure beam**: `ops.model('basic','ndm',2,'ndf',3)`, nodes only at the supports, one `elasticBeamColumn` element per span, a pin and rollers, then `eleLoad('-ele', e, '-type', 'beamUniform', -w)` (downward is negative in the element's local y).
- **Subframe**: nodes at every column-line / grid intersection; columns and beams as `elasticBeamColumn` elements; fixed bases; the line load applied to the beam element. Use `ops.reactions()` then read `nodeReaction(tag, dofs)` for the column forces and compare with the pure-beam reactions.
- **Two limit checks** on any new model before you believe it in between: set the coupling/continuity to *zero* and to *infinite* and confirm you recover the two simple cases; if either fails, nothing in between is worth reading.
- **Speech → model**: the briefing is in words, the file is in code. Keep both. The agent transcribes; you confirm the deck, freeze the loads, and sign the mapping.

Floor Plans 1 (last digit of your student ID is 1, 2, or 3)



Floor Plans 2 (last digit of your student ID is 4, 5, or 6)



Floor Plans 3 (last digit of your student ID is 7, 8, 9, or 0)

